

EPGSpeller: Open-Vocabulary Silent Spelling with Electropalatography

Anonymous Authors

February 19, 2026

Abstract

We study open-vocabulary silent spelling from electropalatography (EPG) and focus on spatial inductive biases derived from electrode layout. We evaluate three EPG front-end representations: (i) a vector encoder that ignores layout, (ii) a layout-aware two-dimensional reconstruction consumed by convolution, and (iii) a layout-aware row/column compression. All reported numbers are generated from pinned evidence files via a deterministic script.

1 Problem setting

Electropalatography measures tongue–palate contact using a fixed electrode array. We treat SmartPalate channel indices as a layout proxy defined by a 16×16 grid map; 120 channels are mapped to grid coordinates and 4 channels are unmapped (see the pinned distribution CSV in our artifact registry). Our decoding target is character sequences, using a CTC-style formulation [1].

2 Models and spatial front-ends

We use a recurrent CTC decoder with a fixed temporal receptive field (kernel length 32 and stride 4) and a shared embedding size 64. We compare the following EPG front-end variants.

Vector (no layout). The vector baseline treats the EPG input as an unordered channel vector.

Layout-aware two-dimensional reconstruction. We scatter the selected channels into the 16×16 grid using the SmartPalate distribution as a proxy layout and apply a two-dimensional convolutional stack to obtain per-frame embeddings.

Row/column compression. We compute row-wise and column-wise pooled summaries from the same reconstructed grid and project them to the shared embedding size.

3 Experimental protocol

We evaluate a within-subject protocol using 4 random splits (seed0, seed1, seed2, seed3) with identical training settings across compared front-ends. Training uses 10000 optimization steps at 10.0 ms frame rate and enables SpecAugment-style masking (SpecAug on: 1) [2].

We focus on two electrode-selection methods for budgeted subsets: TopK (importance-based ranking) and FPS-diverse TopK (importance with spatial diversification).

4 Results

4.1 Two-dimensional convolution vs vector

Table 1 compares vector and two-dimensional convolutional front-ends for multiple electrode budgets. Across the tested budgets, the layout-aware two-dimensional convolutional front-end is worse than the vector baseline in greedy CER and also slower in streaming RTF.

Table 1: Vector vs two-dimensional convolution (mean $\pm$ std over splits).

K	Vector Greedy CER	Spatial2D Greedy CER
TopK		
32	0.1399 $\pm$ 0.0162	0.4885 $\pm$ 0.1269
64	0.1289 $\pm$ 0.0135	0.2609 $\pm$ 0.0300
96	0.1194 $\pm$ 0.0173	0.2304 $\pm$ 0.0350
124	0.1048 $\pm$ 0.0093	0.1831 $\pm$ 0.0103
FPS2K		
32	0.1440 $\pm$ 0.0136	0.5795 $\pm$ 0.3052
64	0.1195 $\pm$ 0.0103	0.3581 $\pm$ 0.3079
96	0.1137 $\pm$ 0.0161	0.3438 $\pm$ 0.3104
124	0.1048 $\pm$ 0.0093	0.1831 $\pm$ 0.0103

Table 2: Vector vs row/column compression (mean $\pm$ std over splits).

K	Vector Greedy CER	RowCol Greedy CER
TopK		
32	0.1399 $\pm$ 0.0162	0.1448 $\pm$ 0.0204
64	0.1289 $\pm$ 0.0135	0.1269 $\pm$ 0.0143
FPS2K		
32	0.1440 $\pm$ 0.0136	0.1466 $\pm$ 0.0080
64	0.1195 $\pm$ 0.0103	0.1255 $\pm$ 0.0193

4.2 Row/column compression vs vector

Table 2 shows that row/column compression stays close to the vector baseline in greedy CER, but increases streaming RTF.

4.3 Spatial augmentation and robustness to electrode failures

To quantify robustness, we apply fixed electrode dropout masks at evaluation time and report the increase in CER relative to the clean setting. Table 3 shows that spatial augmentation reduces the CER degradation under dropout.

Table 3: Fixed-dropout robustness for the two-dimensional front-end (mean $\pm$ std over splits). Values are

Drop rate	No spatial aug	Spatial aug
0.0	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000
0.1	0.002676 $\pm$ 0.000127	0.0183 $\pm$ 0.0065
0.2	0.002895 $\pm$ 0.000304	0.0660 $\pm$ 0.0097
0.3	0.002767 $\pm$ 0.000238	0.1224 $\pm$ 0.0120
0.4	0.002635 $\pm$ 0.000067	

5 Artifacts and auditability

This project maintains a claim-evidence registry in `paper/paper.json`. All numeric facts and result tables in this draft are generated from pinned evidence snapshots via `scripts/paper/generate_facts_text.py`, and a strict checker rejects handwritten numeric literals

References

- [1] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification, 2006.
- [2] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A simple data augmentation method for automatic speech recognition, 2019.